



Banco de Dados

Prof. Elson de Abreu

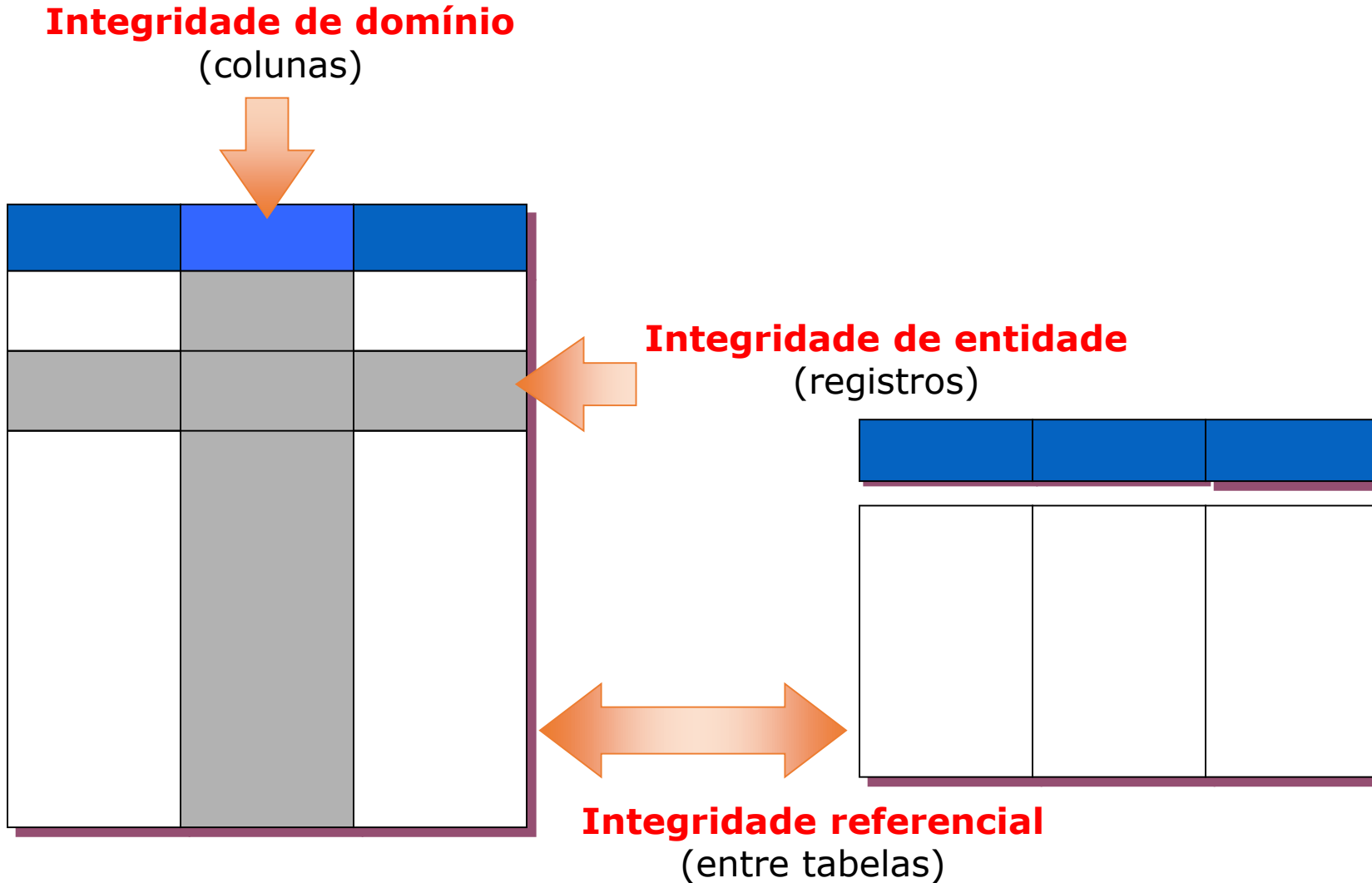


Constraints, subconsultas e views

Constraints

- As *constraints* impõem regras no nível da tabela.
- As *constraints* evitam que uma tabela seja deletada se houver dependências (integridade referencial).
- É possível incluir *constraints* em uma tabela com dados existentes.
- Tipos de restrição válidos no MSSQL Server:
 - NOT NULL
 - UNIQUE
 - PRIMARY KEY
 - FOREIGN KEY
 - CHECK

Tipos de Integridade



Tipo de integridade x tipo de constraint

Tipo de integridade	Tipo de constraint
Domínio	DEFAULT
	CHECK
Entidade	PRIMARY KEY *
	UNIQUE
Referencial	FOREIGN KEY **
	CHECK *

*Não será abordado, consulte a referência bibliográfica.

** Abordado parcialmente.

Constraint Default

- Aplicam-se apenas a instruções INSERT.
- Apenas uma restrição DEFAULT por coluna.
- Permitem alguns valores fornecidos pelo sistema.

```
CREATE TABLE empregados  
(empno INT,  
 nome VARCHAR(100),  
 datacontratacao DATETIME DEFAULT getdate())
```

Ou

```
ALTER TABLE empregados  
ADD CONSTRAINT DF_DATACONTRATAcao DEFAULT  
    getdate()  
FOR datacontratacao
```

Constraint Check (a nível de domínio)

- Usada com as instruções INSERT e UPDATE.
- Pode fazer referência a outra coluna na mesma tabela.
- Não pode conter subconsultas.

```
CREATE TABLE empregados  
(empno INT,  
  nome VARCHAR(100),  
  datanascimento DATETIME CHECK  
    (datanascimento > '01-01-1900' AND datanascimento <  
      getdate())  
)
```

Ou

```
ALTER TABLE empregados  
ADD CONSTRAINT CK_DATANASC  
CHECK (datanascimento > '01-01-1900' AND datanascimento <  
  getdate())  
)
```

Constraint Unique

- Permite um valor nulo.
- São permitidas várias restrições UNIQUE em uma tabela.
- Imposta com um índice exclusivo.

```
CREATE TABLE empregados
```

```
( empno INT,
```

```
  nome VARCHAR(100),
```

```
  cpf CHAR(11)
```

```
CONSTRAINT UK_CPF UNIQUE(cpf))
```

Constraint Foreign Key

- Deve fazer referência a uma restrição PRIMARY KEY ou UNIQUE.
- Fornece uma integridade referencial de uma ou várias colunas.
- Não cria índices automaticamente.
- Os usuários devem ter as permissões SELECT ou REFERENCES em tabelas referenciadas.

Exemplo

```
CREATE TABLE empregados
( empno INT,
  nome VARCHAR(100),
  deptno INT,
  datacontratacao DATETIME,
  CONSTRAINT PK_EMPREGADOS PRIMARY KEY (empno)
)
```

```
ALTER TABLE empregados
ADD CONSTRAINT FK_EMP_DEPT
FOREIGN KEY (deptno) REFERENCES dept (deptno)
ON DELETE CASCADE
ON UPDATE CASCADE
```

- **ON DELETE CASCADE:** Permite exclusão na tabela mãe e das linhas dependentes na tabela filha.
- **ON UPDATE CASCADE:** Permite alteração na tabela mãe e das linhas dependentes na tabela filha.
- *Caso sejam omitidos tais argumentos, o default será aplicado: NO ACTION.*

Domains

- É um tipo de restrição assim como PK's, FK's, Check's.
- Permite que um tipo de dado seja compartilhado por vários atributos.
- São úteis para reunir restrições comuns em campos em um único local para manutenção. Por exemplo, várias tabelas podem conter colunas de endereço de correio eletrônico, todas requerendo a mesma restrição de verificação (CHECK). Em vez de definir as restrições em cada tabela individualmente, pode ser definido um domínio.
- *Este é um conceito generalista, portanto deve-se levar em consideração a possível implementação no SGBD escolhido. No SQL Server 2005 é possível criar um novo tipo de dado do usuário através de "sp_addtype".*

Exemplo

- No SQL Server 2005 a implementação de um domain não é possível através do comando “create domain” igual aos demais SGBD’s. Ao invés disso é possível criar novos tipos de dados através da função `sp_addtype` e adicionar regras através de rules. Para maiores detalhes consulte o manual on-line da ferramenta.

-- criando a regra: maior ou igual a zero.

`create rule R_DMOEDA as`

`@column >= 0`

`go`

-- criando o tipo decimal não nulo

`execute sp_addtype DMOEDA, 'decimal(15,2)', 'not null'`

`go`

-- vinculando a regra ao tipo

`execute sp_bindrule R_DMOEDA, DMOEDA`

`go`

-- utilizando o novo tipo

`CREATE TABLE empregados(`

`empno INT NOT NULL,`

`salario DMOEDA`

`)`

`go`

Subconsultas

- A subconsulta é executada primeiro e o resultado é utilizado pela consulta principal.

SELECT lista_colunas
FROM tabela [,tabela] } Consulta principal

WHERE expressão operador

(**SELECT** lista_colunas
FROM tabela [,tabela])

Subconsultas de uma única linha

- Retornam apenas uma única linha e utilizam os operadores de uma única linha.

Operador	Significado
=	Igual a
>	Maior do que
>=	Maior ou igual a
<	Menor do que
<=	Menor ou igual a
<>	Diferente de

Exemplo

- Quais as fitas possuem o mesmo gênero de “O Sexto Sentido” ?

```
SELECT codfita, nome  
FROM fita  
WHERE codgenero = (SELECT codgenero FROM fita WHERE nome = 'O sexto sentido')
```

codfita nome

240 À espera de um milagre

250 O sexto sentido

(2 row(s) affected)

Subconsultas de várias linhas

- Retornam várias linhas e utilizam operadores de comparação de várias linhas.

Operador	Significado
IN	Igual a pelo menos um elemento da lista.
ANY	A comparação deve ser satisfeita para pelo menos um elemento da lista.
ALL	A comparação deve ser satisfeita para todos os elementos da lista.

Exemplo

- Para os exemplos de subconsulta a seguir utilizaremos as seguintes tabelas:

Produtos

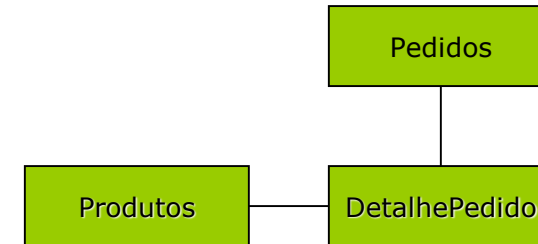
IDPRODUTO	DESCRICAO	PRECOUNIDADE
1	TV Plasma 42"	2500.00
2	DVD Player	250.00
3	Micro computador	2000.00
4	TV 29"	800.00
5	Micro system	1700.00
6	Câmera Digital 7 MP	800.00
7	Telefone sem fio	120.00

(7 row(s) affected)

DetalhePedido

IDPEDIDO	IDPRODUTO	QUANTIDADE	PRECOUNIDADE	DESCONTO
1	1	1	1875.00	0.25
1	3	2	1500.00	0.25
2	4	1	800.00	0.00
2	6	1	720.00	0.10
3	1	1	1875.00	0.25
3	7	2	108.00	0.10
4	5	1	1615.00	0.05
5	2	2	225.00	0.10
5	3	3	1500.00	0.25

(9 row(s) affected)



Exemplo: “IN”

- Listar todos os produtos que tiveram um desconto maior ou igual a 10 por cento sobre o valor unitário:

SELECT * **FROM** Produtos

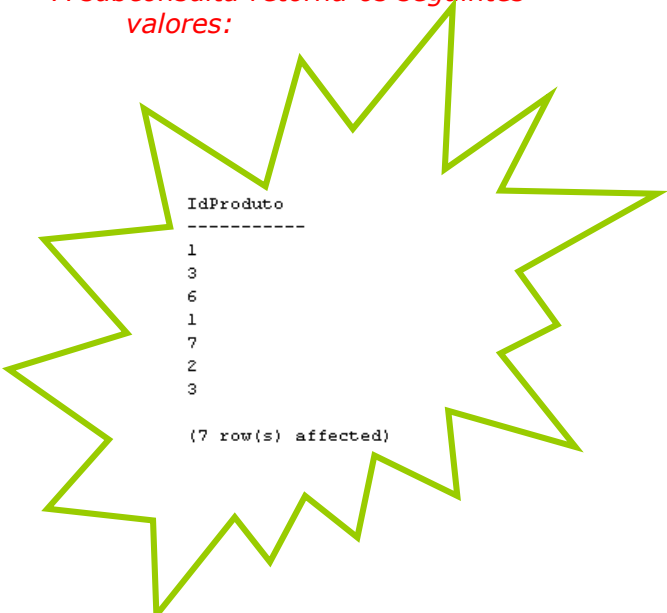
WHERE IdProduto **IN** (**SELECT** IdProduto **FROM** DetalhePedido **WHERE** Desconto >= 0.10)

IDPRODUTO	DESCRICAO	PRECOUNIDADE
1	TV Plasma 42"	2500.00
2	DVD Player	250.00
3	Micro computador	2000.00
6	Câmera Digital 7 MP	800.00
7	Telefone sem fio	120.00

(5 row(s) affected)

Lembrete!

A subconsulta retorna os seguintes valores:



IdProduto

1
3
6
1
7
2
3

(7 row(s) affected)

Exemplo: “ANY”

- Listar todos os produtos cujo preço unitário seja maior que **pelo menos** um produto vendido com desconto maior ou igual a 25 por cento:

SELECT * **FROM** Produtos

WHERE PrecoUnidade > **ANY** (**SELECT** PrecoUnidade **FROM** DetalhePedido **WHERE** Desconto >= 0.25)

IDPRODUTO	DESCRICAO	PRECOUNIDADE
1	TV Plasma 42"	2500.00
3	Micro computador	2000.00
5	Micro system	1700.00

(3 row(s) affected)

Lembrete!
A subconsulta retorna os seguintes valores:



PrecoUnidade
1875.00
1500.00
1875.00
1500.00

(4 row(s) affected)

Obs.: a operação “= ANY” equivale ao operador “IN”, devolvendo o mesmo resultado.

Exemplo: “ALL”

- A consulta devolverá aqueles produtos cujo preço unitário seja maior que o de todos os produtos vendidos com um desconto igual ou maior ao 25 por cento.

SELECT * **FROM** Produtos

WHERE PreçoUnidade > **ALL**

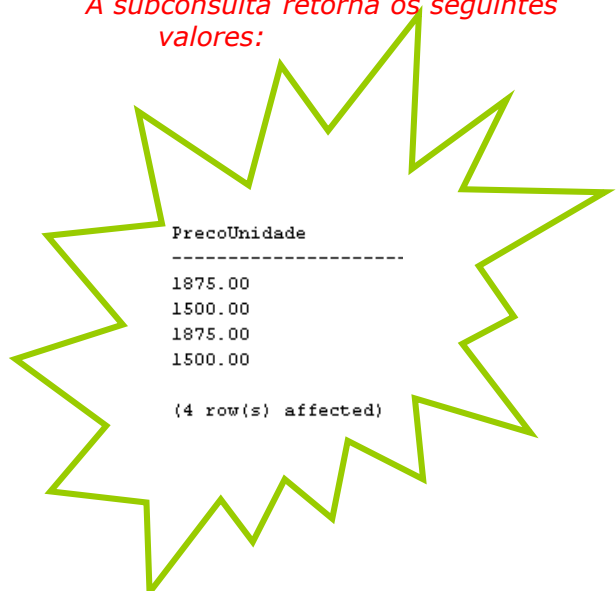
(**SELECT** PreçoUnidade **FROM** DetalhePedido **WHERE** Desconto >= 0.25)

IDPRODUTO	DESCRICAO	PRECOUNIDADE
1	TV Plasma 42"	2500.00
3	Micro computador	2000.00

(2 row(s) affected)

Lembrete!

A subconsulta retorna os seguintes valores:



```
PreçoUnidade
-----
1875.00
1500.00
1875.00
1500.00

(4 row(s) affected)
```

Outras utilizações de subconsultas

- Na cláusula FROM: o resultado da subconsulta é tratado como uma tabela.

```
SELECT U.IDUSUARIO,  
       U.NOME,  
       B.NOME AS AG_BANCARIA  
FROM USUARIOS U, (SELECT B.IDBANCO, B.NOME FROM BANCOS B) AS B  
WHERE B.IDBANCO = U.IDBANCO
```

- Na cláusula SELECT: o resultado da subconsulta pode ser tratado como uma expressão.

```
SELECT IDUSUARIO,  
       NOME,  
       AG_BANCARIA = (SELECT B.NOME FROM BANCOS B WHERE B.IDBANCO = U.IDBANCO)  
FROM USUARIOS U
```

Views

- Uma *view* é uma tabela virtual cujos dados são definidos por uma consulta. O resultado de uma *view* é produzido dinamicamente quando é referenciada (uma tabela temporária é criada fisicamente a partir da primeira consulta).
- Uma *view* pode limitar o acesso a dados ou simplesmente exibir os dados importantes de uma ou várias tabelas, ocultando a complexidade de um banco de dados.
- Uma visão para ser atualizável não pode referenciar mais de uma tabela, seja através de comandos **UNION** ou utilizando junção, não pode usar funções agregadas e precisa incluir todos os campos **not null** que não apresentam valor default.
- Restrições às definições:
 - Não pode incluir a instrução ORDER BY
 - Não pode incluir a palavra-chave INTO

* O comando **SELECT INTO** cria uma nova tabela com base no resultado de uma query.

Exemplo

Forma:

```
CREATE VIEW nome ( [colunas] ) AS  
consulta
```

Obs: "colunas" é uma lista de parâmetros opcional. É utilizada para renomear as colunas da consulta.

Definindo:

```
CREATE VIEW v_fitos AS  
SELECT codfita, nome FROM fitas
```

Referenciando:

```
SELECT * FROM v_fitos
```